

Homework 19: A Download Service

Measure the difference yourself, then find the rate limit you removed without noticing.

Part One: Measure It

Create `DownloadService.java` in the classic form.

Run 2,000 simulated downloads, each blocking for 100ms, three ways:

1. `newVirtualThreadPerTaskExecutor()`
2. `newFixedThreadPool(100)`
3. `newFixedThreadPool(25)`

Print all three timings.

Then **predict** the two platform numbers before looking at them. The formula is simple, and it comes from how many tasks can be blocked at once.

Print your predicted floors next to the measurements.

Explain It

A comment covering:

- Why the platform numbers are `(tasks / poolSize) × latency`
- Why the virtual number has no `poolSize` in it
- **The actual point.** The platform timings depend on a number you had to guess. Say what guessing it too low costs, and what guessing it too high costs.

Part Two: The Limit You Removed

A downstream service tolerates **5 concurrent calls**. Your old code used `newFixedThreadPool(5)`, so that limit was enforced by accident.

Write a `ConcurrencyTracker` that records the **peak** number of calls in flight at once. Then run 40 calls three ways and print the peak each time:

1. Platform pool of 5
2. Virtual threads, **no other change**
3. Virtual threads plus an explicit `Semaphore(5)`

The second one will hit 40. Nothing will fail, nothing will warn.

Your tracker's peak must itself be race-free. Two threads raising the peak at the same moment must not lose an update, and working out how to do that is part of the exercise.

Explain It

A comment covering:

- What two separate decisions the old pool size was making at once
- Why the `Semaphore` is better than the pool, even though both give 5
- **Where `release()` must go, and what happens if it is skipped on an exception.** Be specific about the failure mode

Part Three: Scoped Values

Bind a request id with `ScopedValue`, then read it **two frames deep** without passing it as a parameter.

Show that it is unbound outside the scope.

Then start a child thread inside the scope and show whether it sees the binding.

Explain It

A comment on:

- What you did **not** have to write, compared to a `ThreadLocal`
- What a `ThreadLocal` would still hold after the request finished, and why that matters when the thread is reused
- Why a child thread does not inherit the binding, and what would be needed for it to

Part Four: What Did Not Change

Run four virtual threads incrementing a plain `int` field 100,000 times each.

Print expected against actual.

Write a comment on why virtual threads did not help here, and on whether the risk of this bug went **up or down** now that spawning ten thousand threads is easy.

Acceptance Criteria

- ☐ Runs with `java DownloadService.java`
- ☐ Three timings printed, with predicted floors alongside
- ☐ The explanation covers both directions of guessing the pool size wrong

- ☐ `ConcurrencyTracker` records a peak and is itself race-free
- ☐ The three peaks are 5, 40 and 5
- ☐ A comment names the two decisions the pool size was conflating
- ☐ `release()` is in a `finally`, with a comment on the leak if it is not
- ☐ A scoped value is read two frames deep with no parameter passing
- ☐ It is shown unbound outside the scope
- ☐ The child-thread behaviour is shown and explained
- ☐ The unsafe counter is shown losing increments under virtual threads

Stretch

Replace the `Semaphore` with a bounded queue: a producer submitting work and a fixed number of virtual-thread consumers draining it.

Then write a comment comparing the two. Which one applies backpressure to the submitter? What happens under each when work arrives faster than it can be processed, and which failure would you rather debug?

Hint, if your tracker's peak looks too low

`peak = Math.max(peak, current)` is a read-modify-write, which is Module 18's race with different variable names. `AtomicInteger` has a method that retries until it wins; find it.